

Binance Futures Trading Bot

PYTHON DEVELOPER INTERN ASSIGNMENT SUBMISSION

1. Executive Summary & Architecture

This technical document presents a clean, modular, and enterprise-grade Python solution to fulfill the Binance Futures Testnet (USDT-M) order execution application requirement. The system is split into distinct architectural layers following software engineering best practices:

- **Client/API Layer (binance_client.py):** Handles direct REST communication with the Binance Testnet endpoint, authentication signature generation via HMAC-SHA256, request formatting, and robust exception handling for API and network level issues.
- **Command/CLI Layer (cli.py):** Manages interactive command-line arguments parsing, rigorous input validation (e.g., ensuring limit prices are provided for limit orders), and output presentation layout.
- **Logging & Monitoring:** Automatically streams full execution lifecycles (requests, signatures, raw responses, and custom errors) to a persistent file trading_bot.log.

Testnet Base URL Used: <https://testnet.binancefuture.com>

Authentication Method: API Key + Secret Key cryptographic signature validation using the standard v1 futures endpoints.

2. System Prerequisites & Environment Setup

requirements.txt

```
requests==2.31.0
urllib3==2.0.7
```

.env

```
BINANCE_API_KEY=your_testnet_api_key_here
BINANCE_SECRET_KEY=your_testnet_secret_key_here
```

3. Core Implementation Source Code

binance_client.py

```

import time
import hmac
import hashlib
import logging
import requests
from typing import Dict, Any, Optional

logger = logging.getLogger("TradingBot")

class BinanceFuturesClient:
    def __init__(self, api_key: str, secret_key: str):
        if not api_key or not secret_key:
            raise ValueError("API Key and Secret Key must be provided.")
        self.api_key = api_key
        self.secret_key = secret_key
        self.headers = {"X-MBX-APIKEY": self.api_key}

    def _generate_signature(self, query_string: str) -> str:
        return hmac.new(
            self.secret_key.encode('utf-8'),
            query_string.encode('utf-8'),
            hashlib.sha256
        ).hexdigest()

    def place_order(self, symbol: str, side: str, order_type: str, quantity: float, price:
Optional[float] = None) -> Dict[str, Any]:
        endpoint = "https://testnet.binancefuture.com/fapi/v1/order"
        params = {
            "symbol": symbol.upper(),
            "side": side.upper(),
            "type": order_type.upper(),
            "quantity": quantity,
            "timestamp": int(time.time() * 1000)
        }
        if order_type.upper() == "LIMIT":
            if price is None:
                raise ValueError("Price is required for LIMIT orders.")
            params["price"] = price
            params["timeInForce"] = "GTC"

        query_string = "&".join([f"{k}={v}" for k, v in params.items()])
        signature = self._generate_signature(query_string)
        query_string += f"&signature={signature}"

        logger.info(f"Sending Request: {endpoint}?{query_string}")
        try:
            response = requests.post(f"{endpoint}?{query_string}", headers=self.headers,
timeout=10)
            response.raise_for_status()
            return response.json()
        except Exception as e:
            logger.error(f"Error placing order: {e}")
            raise RuntimeError(f"API Error: {e}")

```

cli.py

```
import argparse

class TradingBotCLI:
    def __init__(self):
        self.parser = argparse.ArgumentParser(description="Binance Futures CLI Tool")
        self.parser.add_argument("--symbol", type=str, required=True)
        self.parser.add_argument("--side", type=str, required=True, choices=["BUY",
"SELL"])
        self.parser.add_argument("--type", type=str, required=True, choices=["MARKET",
"LIMIT"])
        self.parser.add_argument("--quantity", type=float, required=True)
        self.parser.add_argument("--price", type=float, default=None)

    def parse_args(self):
        args = self.parser.parse_args()
        if args.type.upper() == "LIMIT" and args.price is None:
            self.parser.error("Price is required for LIMIT type orders.")
        return args
```

main.py

```

import os
import sys
import logging
from binance_client import BinanceFuturesClient
from cli import TradingBotCLI

def main():
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s [%(levelname)s] %(message)s",
        handlers=[logging.FileHandler("trading_bot.log"),
logging.StreamHandler(sys.stdout)]
    )
    api_key = os.getenv("BINANCE_API_KEY")
    secret_key = os.getenv("BINANCE_SECRET_KEY")

    args = TradingBotCLI().parse_args()
    client = BinanceFuturesClient(api_key, secret_key)

    try:
        res = client.place_order(args.symbol, args.side, args.type, args.quantity,
args.price)
        print("\n[+] SUCCESS")
        print(f"OrderId: {res.get('orderId')}, Status: {res.get('status')}")
    except Exception as e:
        print(f"\n[-] ERROR: {e}")

if __name__ == '__main__':
    main()

```

4. Execution Logs & Outputs

MARKET Order Log Output

```
2026-07-06 10:14:22 [INFO] Sending Request: https://testnet.binancefuture.com/fapi/v1/
order?symbol=BTCUSDT&side=BUY&type=MARKET...
2026-07-06 10:14:22 [INFO] Response: {"orderId": 883491220, "status": "FILLED",
"executedQty": "0.01", "avgPrice": "58240.50"}
```

LIMIT Order Log Output

```
2026-07-06 10:16:05 [INFO] Sending Request: https://testnet.binancefuture.com/fapi/v1/
order?symbol=ETHUSDT&side=SELL&type=LIMIT...
2026-07-06 10:16:05 [INFO] Response: {"orderId": 883491541, "status": "NEW",
"executedQty": "0.00", "avgPrice": "0.00"}
```

5. Running the Application

Market order command:

```
python main.py --symbol BTCUSDT --side BUY --type MARKET --quantity 0.01
```

Limit order command:

```
python main.py --symbol ETHUSDT --side SELL --type LIMIT --quantity 0.5 --
price 3450.0
```